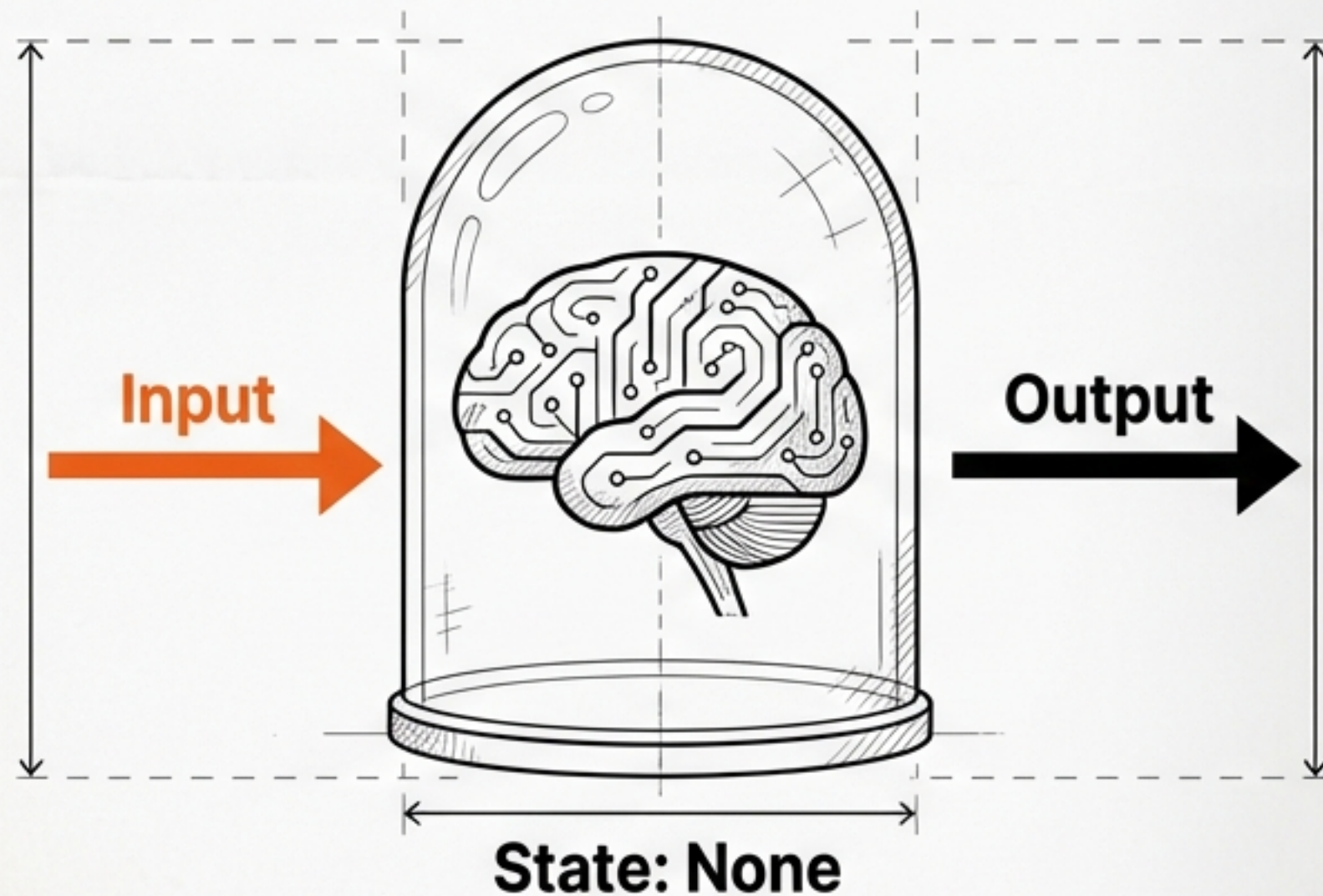


At this stage, the model is a 'brain in a jar'—pure processing power. It receives input, fires a synapse, and returns output. It has no memory of the past and no hands to touch the world. It is isolated, synchronous, and ephemeral.



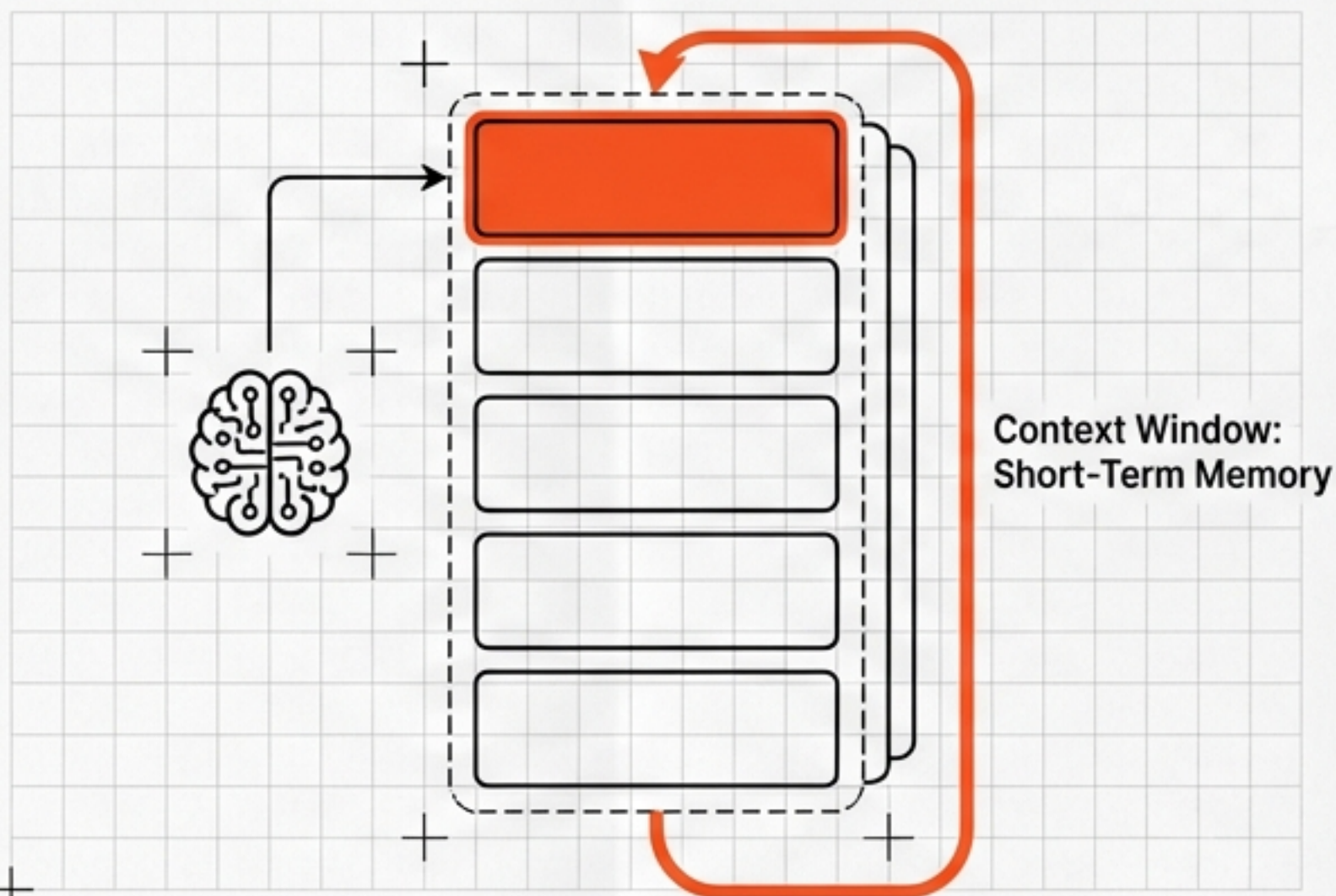
```
from openai import OpenAI
client = OpenAI(api_key="sk-...")

# 1. The Spark: A stateless request
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "user", "content": "Hello,
        world! Who are you?"}
    ]
)

# The brain fires a single synapse
print(response.choices[0].message.content)
```


The Memory: Context Loop

Intelligence requires continuity. Without memory, every "hello" is the first. We manually construct a "Context Window" by maintaining a list of messages. We become the keepers of state, feeding the model its own history with every new turn to create the illusion of time.



02_manual_history.py

```
# 2. The Memory: Accumulating context
```

```
history = []
```

```
while True:
```

```
    user_input = input("You: ")
```

```
    # We manually add the new user message to the stack
```

```
    history.append({"role": "user", "content": user_input})
```

```
    response = client.chat.completions.create(
```

```
        model="gpt-4o",
```

```
        messages=history # The entire life history is passed  
    )    every time
```

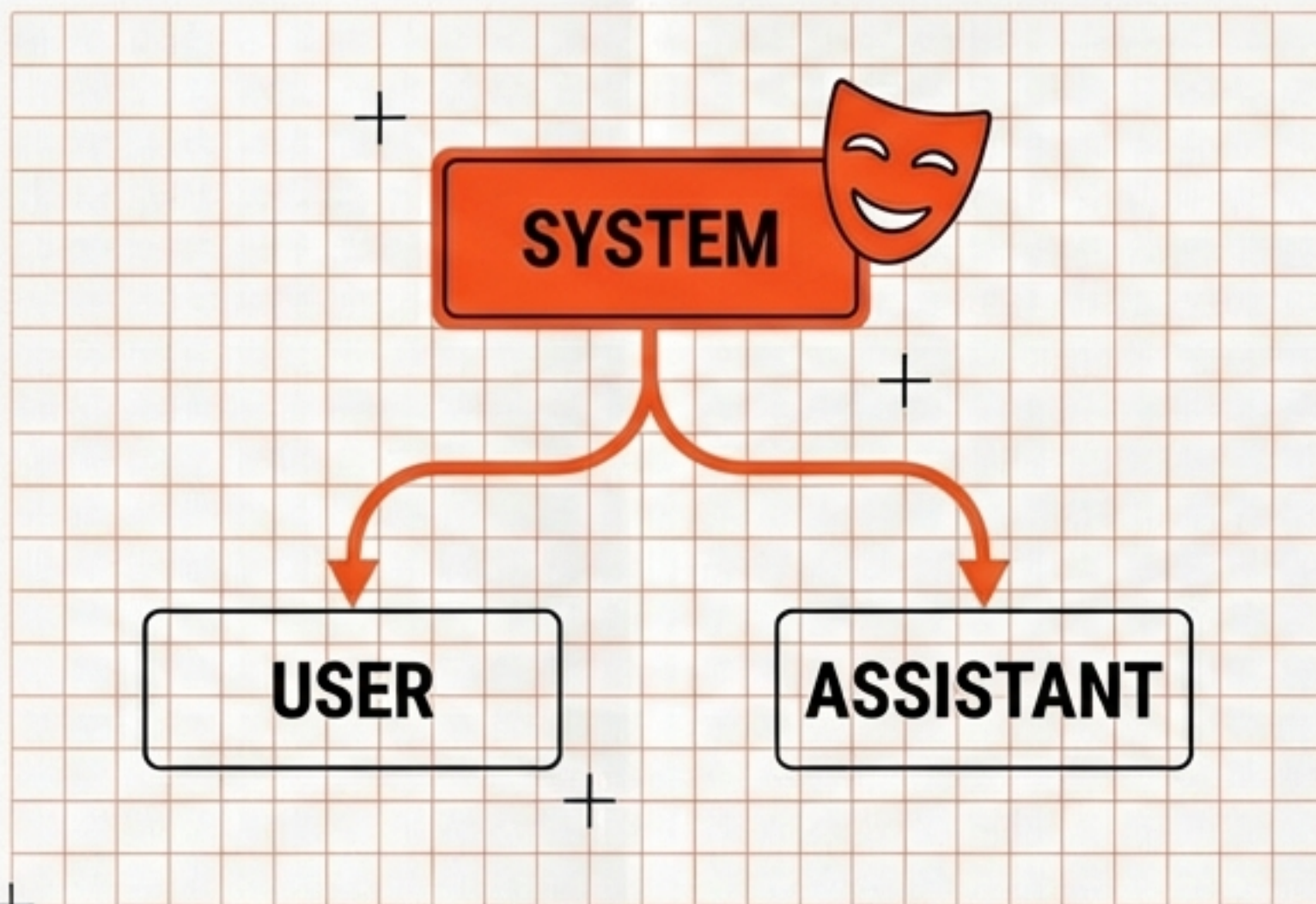
```
    reply = response.choices[0].message.content
```

```
    history.append({"role": "assistant", "content": reply})
```

```
    print(f"AI: {reply}")
```


The Persona: System Roles

Memory gives continuity; the System Role gives character. This is the 'Prime Directive.' By injecting high-level instructions at the very top of the stack, we steer the model's behavior, tone, and constraints before the user even speaks.



03_system_persona.py

```
# 3. The Personality: Defining the Prime Directive
messages = [
    {
        "role": "system",
        "content": "You are a stoic philosopher. Answer briefly."
    },
    {"role": "user", "content": "I lost my keys."}
]

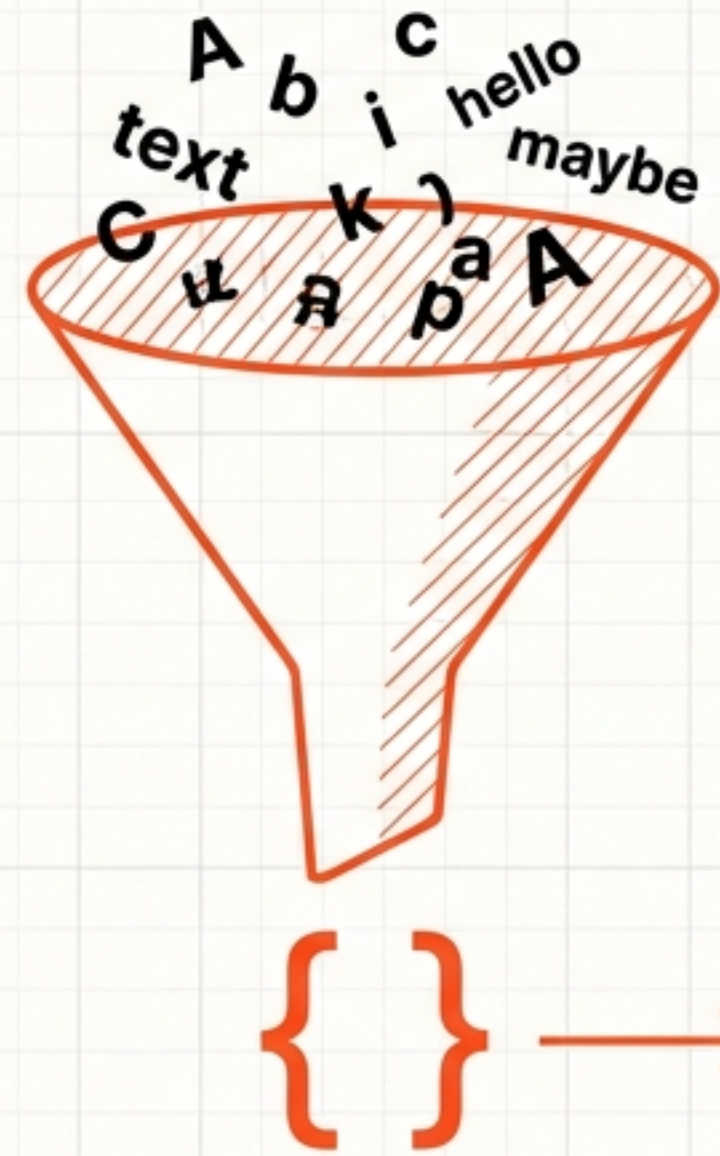
response = client.chat.completions.create(
    model="gpt-4o",
    messages=messages
)

# Output: "Loss is inevitable. Attachment is the source of suffering."
print(response.choices[0].message.content)
```


The Translator: Structured Output

Software runs on rigid data, not fuzzy prose. To integrate AI into code, we must force it to speak JSON.

This layer acts as a funnel, collapsing the probabilistic nature of the LLM into deterministic, machine-readable structures.



Fuzzy Logic → JSON Mode

04_json_mode.py

4. The Translator: Forcing machine-readable output

```
response = client.chat.completions.create(
    model="gpt-4o",
    response_format={"type": "json_object"}, # Enforcing the structure
    messages=[
        {"role": "system", "content": "Output JSON only."},
        {"role": "user", "content": "List 3 colors and their hex codes."}
    ]
)

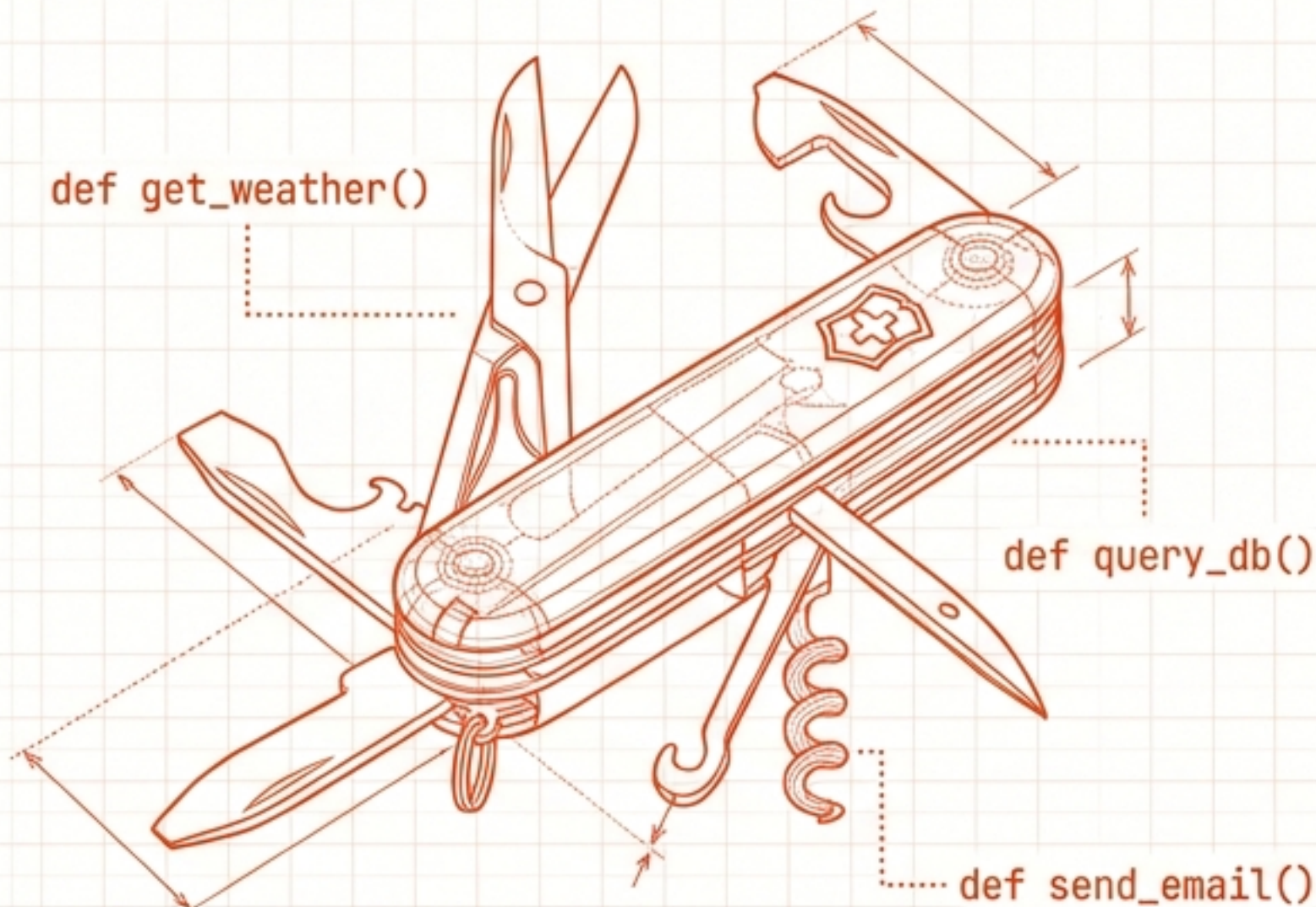
import json

data = json.loads(response.choices[0].message.content)

print(data['colors'][0]['hex']) # The AI now speaks 'Code'
```


The Hands: Defining Tools

An observer becomes an agent when it gains tools. We describe functions—like weather sensors or database queries—to the model using JSON schemas. We don't execute them yet; we simply teach the brain that it has hands.



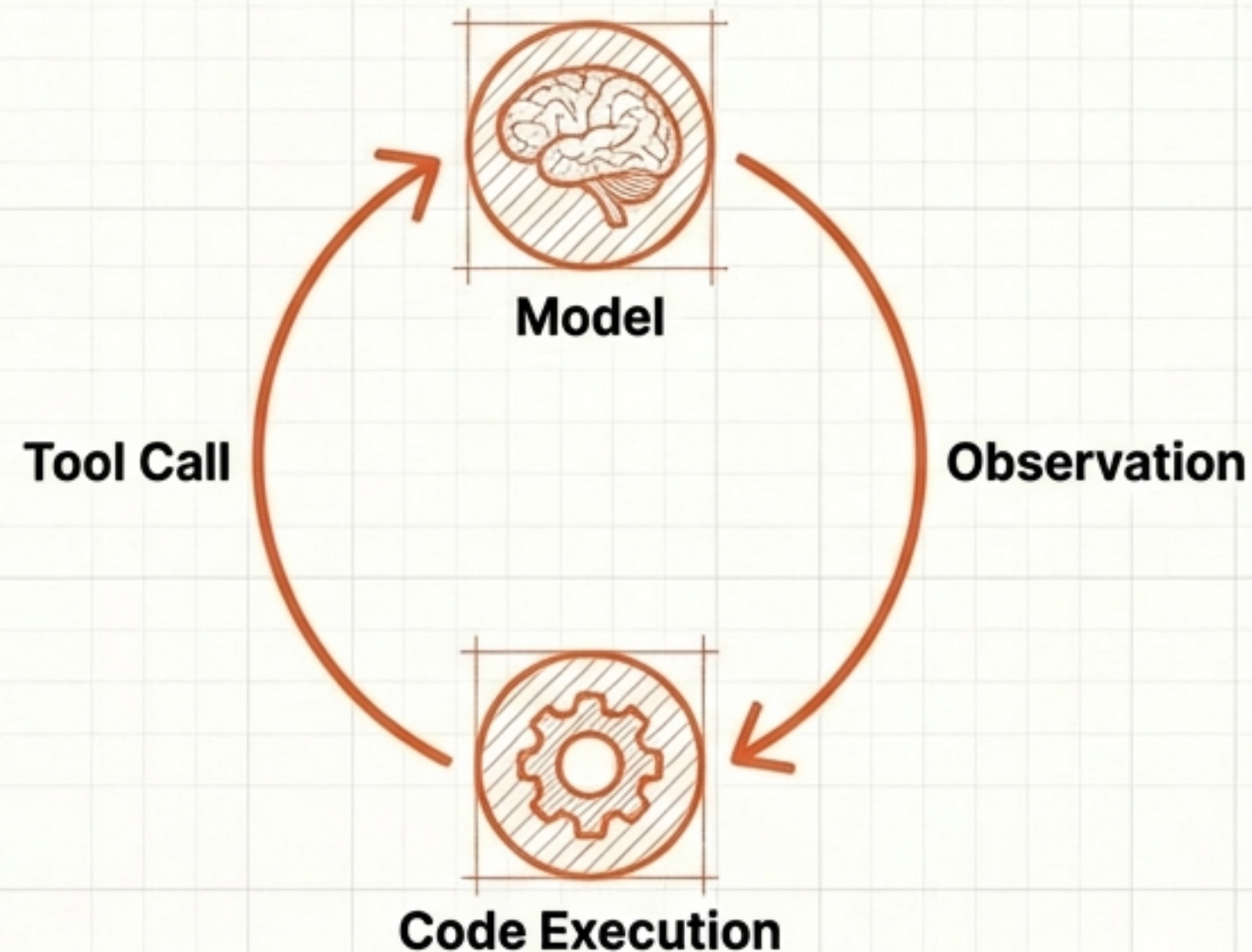
05_tool_definitions.py

```
# 5. The Hands: Defining tools the model can "see"
tools = [{
    "type": "function",
    "function": {
        "name": "get_weather",
        "description": "Get current temperature",
        "parameters": {
            "type": "object",
            "properties": {
                "location": {"type": "string"}
            },
            "required": ["location"]
        }
    }
}]
```

```
# We pass the definition (schema) to the model.
# The model decides IF it needs to use the tool.
```


The Action: Execution Loop

This is the mechanical heart of an agent. The model requests a tool; we execute the code; we feed the result back. This request-execute-observe cycle allows the AI to manipulate the real world.



06_tool_loop.py

```
# 6. The Action: The Execution Loop
response = client.chat.completions.create(
    model="gpt-4o", messages=history, tools=tools
)
msg = response.choices[0].message

if msg.tool_calls:
    # 1. The Brain decides to act
    call = msg.tool_calls[0]

    # 2. The Body executes (Mock function)
    loc = json.loads(call.arguments)['location']
    result = get_weather(loc)

    # 3. The Observation is fed back
    history.append(msg)
    history.append({
        "role": "tool",
        "tool_call_id": call.id,
        "content": result
    })
```


The Library: Simple RAG

Training data is frozen in time. Retrieval Augmented Generation (RAG) gives the model eyes to read the present. We inject relevant documents into the context window just in time, grounding the model in reality and reducing hallucinations.



07_simple_rag.py

```
# 7. The Library: Injecting Knowledge (RAG)
user_query = "What is our refund policy?"
```

```
# Simulate retrieving a document chunk from a Vector DB
retrieved_doc = search_knowledge_base(user_query)
```

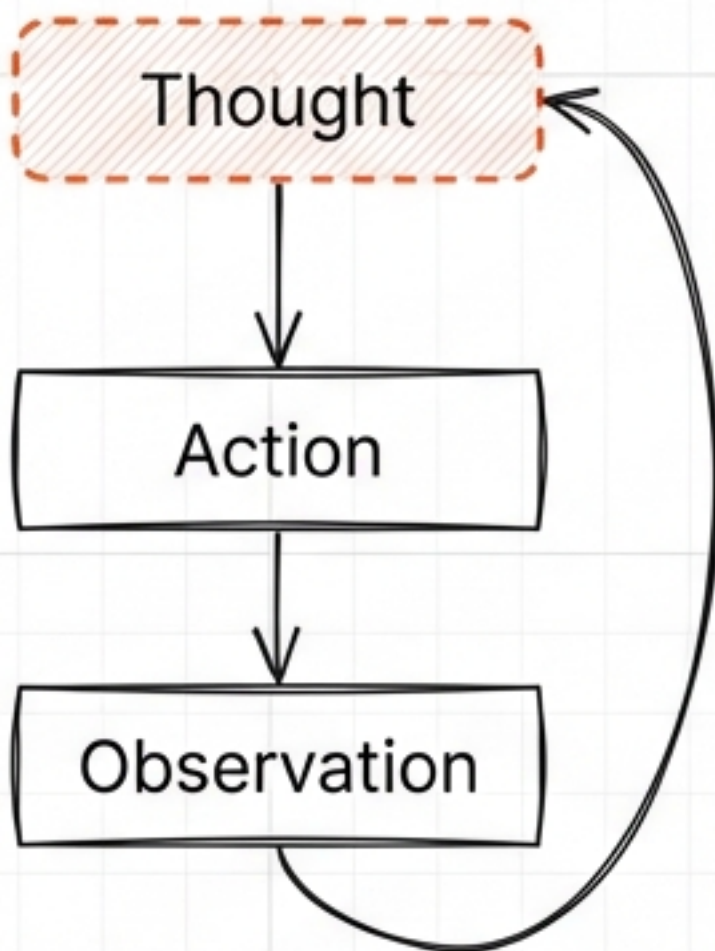
```
messages = [
    # Injecting the knowledge into the system prompt
    {"role": "system", "content": "Answer using the
    context below."},
    {"role": "system", "content": f"Context:
    {retrieved_doc}"},
    {"role": "user", "content": user_query}
]
```

```
response = client.chat.completions.create(
    model="gpt-4o", messages=messages
)
```

```
# The model now "knows" the refund policy.
```


The Reasoning: ReAct Pattern

Complex tasks require an inner monologue. The ReAct (Reason + Act) pattern forces the model to verbalize its plan before acting. This explicit 'thinking' step allows for self-correction and multi-step problem solving.



08_react_pattern.py

```
# 8. The Reasoning: ReAct Pattern
react_prompt = """
Answer the user using this format:
Thought: [Your reasoning]
Action: [Tool to use]
Observation: [Result of tool]
"""

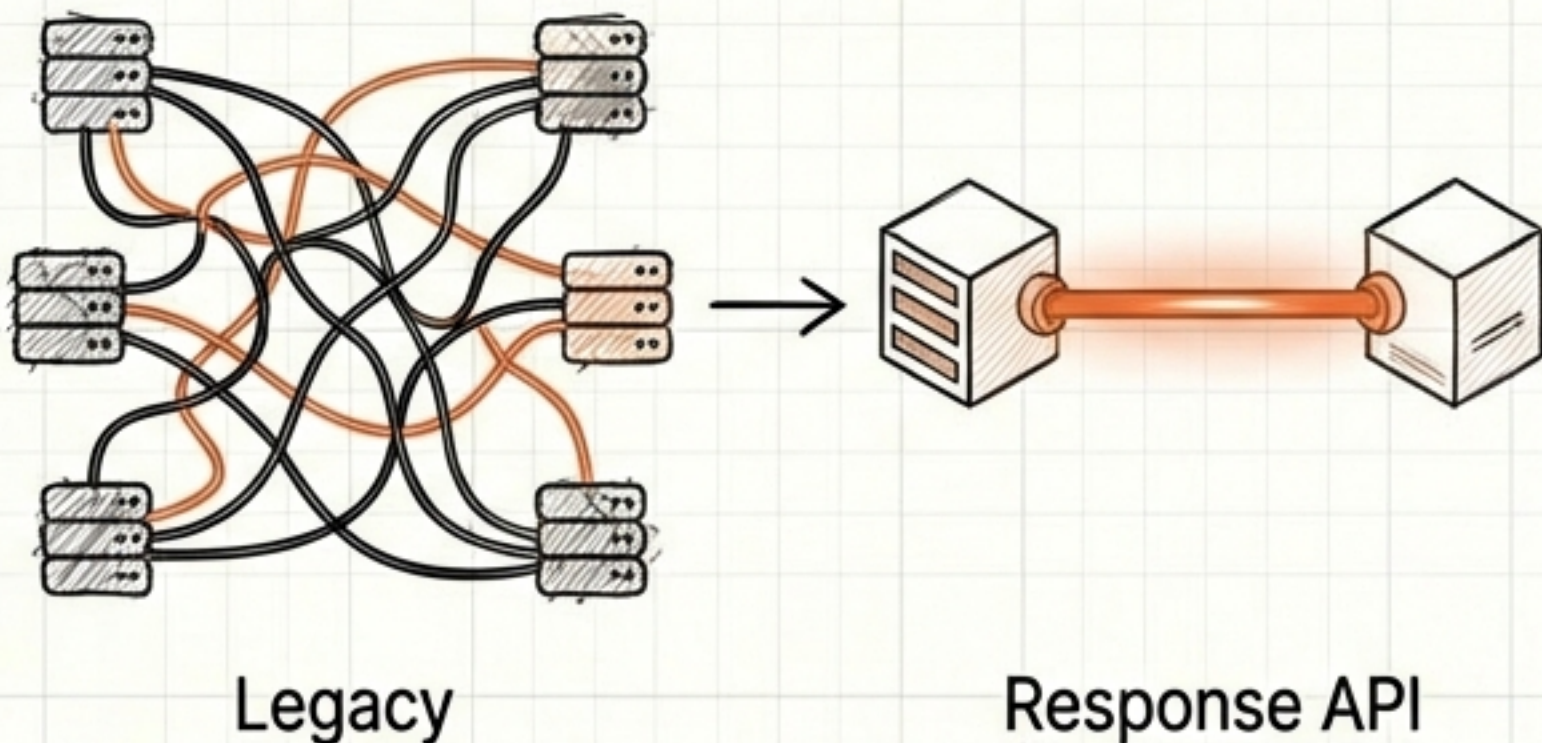
history.append({"role": "system", "content": react_prompt})

# The model now "thinks" before acting:
# Thought: I need to check the weather first.
# Action: get_weather("London")
# Observation: 15°C, Rainy
# Thought: The user needs an umbrella.
```


The Evolution: Response API

Managing loops and lists manually is brittle. The new Response API internalizes state management. We stop passing message lists and start passing a "previous_response_id". The platform now holds the thread, streamlining the architecture.

Stateless vs Native State



09_response_api.py

```
# 9. The Evolution: Native State Management
# No more manual history lists.

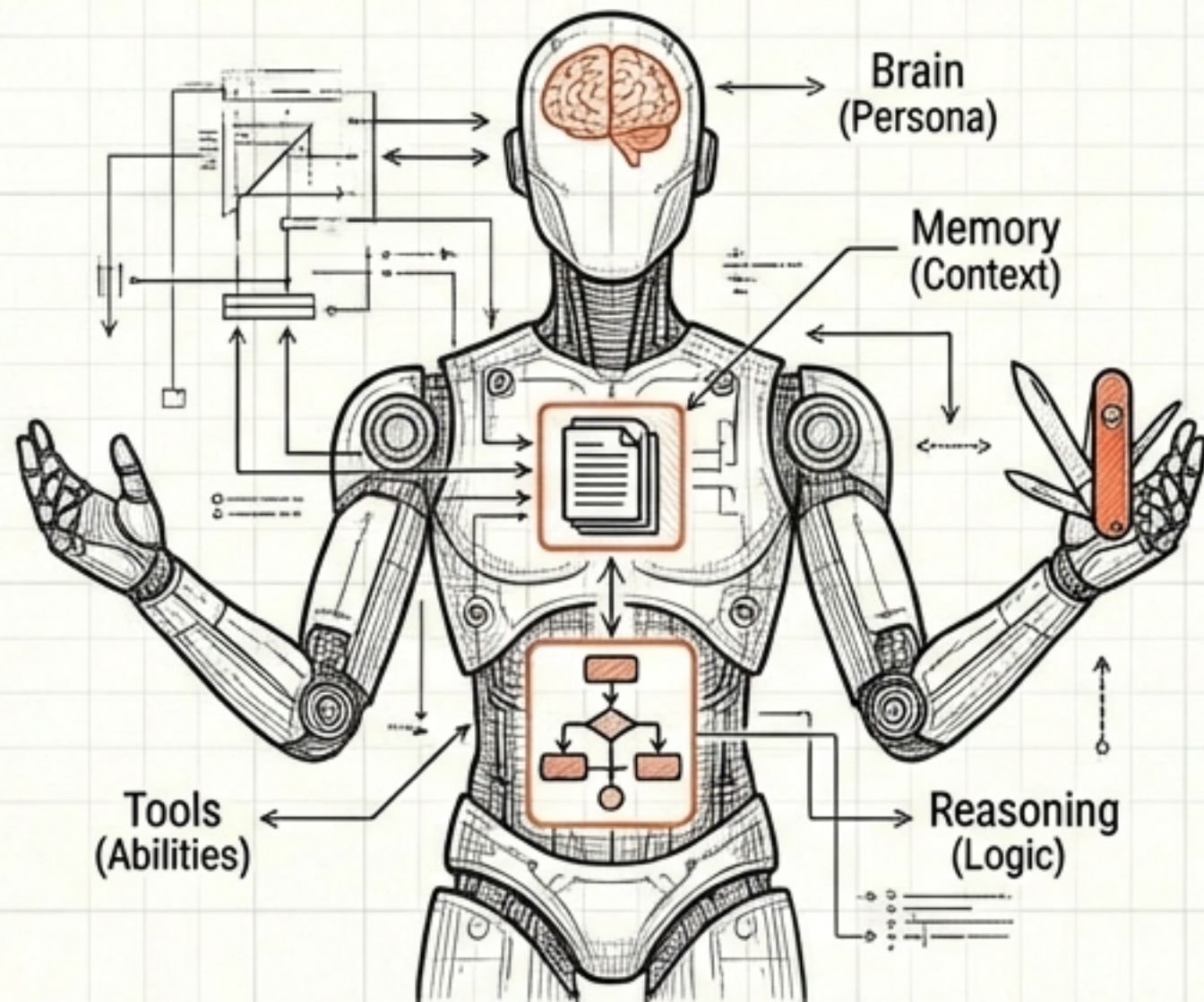
# Turn 1: Initial Prompt
resp1 = client.responses.create(
    model="gpt-4o",
    input="My name is Sarah."
)

# Turn 2: Automatic Context Awareness
# We just reference the ID of the previous turn.
resp2 = client.responses.create(
    model="gpt-4o",
    input="What is my name?",
    previous_response_id=resp1.id # API remembers "Sarah"
)

print(resp2.output_text)
```


The Lifeform: Agent Class

We combine all components—Memory, Persona, Tools, and Reasoning—into a single ‘Lifeform.’ The Agent Class encapsulates the complexity, creating a reusable, autonomous entity capable of navigating the world.



10_simple_agent.py

```
# 10. The Lifeform: A Simple Agent Class
class SimpleAgent:
    def __init__(self, system_prompt, tools):
        self.history = [{"role": "system", "content": system_prompt}]
        self.tools = tools

    def chat(self, user_input):
        self.history.append({"role": "user", "content": user_input})
        # ... logic to handle API call & tool loop ...
        return final_response

# Instantiating the lifeform
agent = SimpleAgent(
    system_prompt="You are a researcher.",
    tools=[web_search]
)
agent.chat("Find the latest news on Agentic AI.")
```